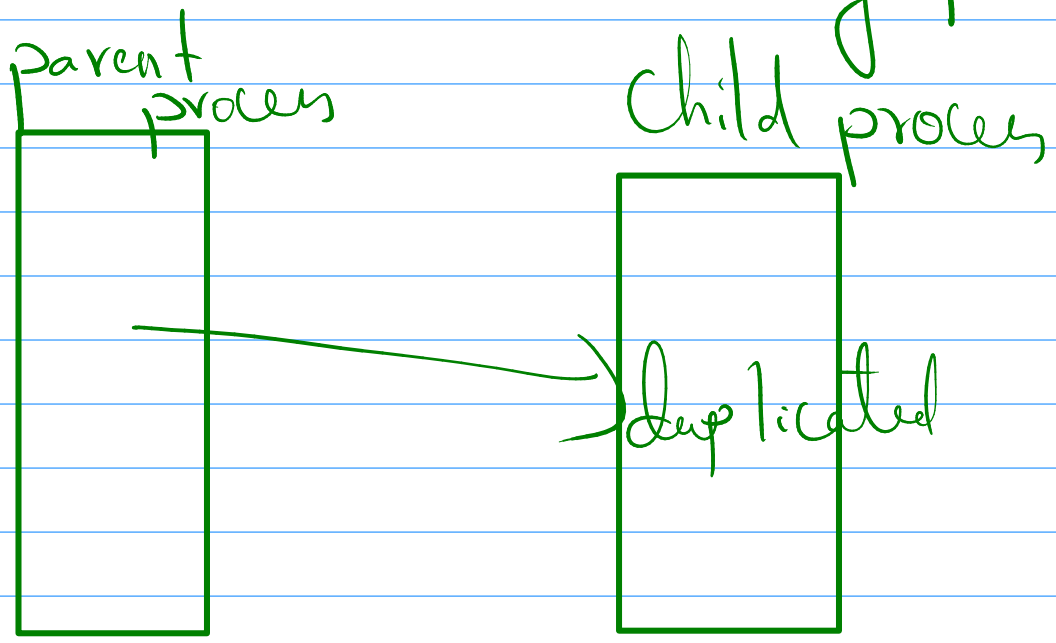
fork → Creating a process

↳ { 0 → child process
PID → parent "
of child
process

→ On fail → -1 to the calling process.



PC, Text, Heap, Data, Stack
↓

PID, PPID

Record locks, Memory locks → not inherited

→

```
main( )
{
    fork( );
    Pf(" ");
    perror(" ");
}
```

errno = 4

```
main( )
{
    if(prime(n))
        etu prime
    else
        not prime
}
```

```
int prime (int x)
{
    ....
    if (prime)
        return 1
    else return 0
}
```

```
main()
{
        int ret=fork();
        if(ret==-1)
        {
                perror("fork");
                exit(0);
        }
        if(ret)
        {
                printf("I am in parent process with PID=%d PPId=%d\
        }
        else
        {
                printf("I am in child process with PID=%d PPId=%d\n"
        }
}
```

If fails ret = -1

process will terminate

```
main()
{
    int ret=fork();
    if(ret==-1)
    {
        perror("fork");
        exit(0);
    }
    if(ret)
    {
        printf("I am in parent process with PID=%d PPId=%d\n"
    }
    else
    {
        printf("I am in child process with PID=%d PPId=%d\n",g
    }
}
```

In parent procey

ret = non zero

parent procey gets terminate

```
main()
{
    int ret=fork();
    if(ret==-1)
    {
        perror("fork");
        exit(0);
    }
    if(ret)
    {
        printf("I am in parent process with PID=%d PPId=%d\n",
    }
    else
    {
        printf("I am in child process with PID=%d PPId=%d\n",ge
    }
}
```

→ In child process ret = 0

→ child process gets terminated.

Parent process & child process are running Simultaneously.

Q How to make if & else (both the blocks) to execute

A1 Using goto

A2 Using fork → Simultaneously

sleep vs delay_s

main()
≡
sleep(10)

waiting state → cpu is not with this job

↳ uc

delay_s(10);

pc → void delay_s(int t)
≡
≡